

MILESTONE 4

ExoHabitAI

Exoplanet Habitability Prediction System

Deployment & Documentation

PROJECT	INTERN	PLATFORM	DATE
ExoHabitAI	Shishir T S	Render	April 2026

Python 3.10	Flask 3.0	XGBoost	scikit-learn 1.5.1	Render
-------------	-----------	---------	--------------------	--------

Modules Covered

Module 7	Module 8
Visualization & Dashboard	Deployment & Documentation

TABLE OF CONTENTS

1.	Project Overview	3
2.	Module 7 — Visualization & Dashboard	3
3.	Module 8 — Deployment & Documentation	4
4.	Flask Backend Architecture	4
5.	API Endpoints Reference	5
6.	Frontend Architecture	6
7.	ML Model Integration	7
8.	Deployment on Render	7
9.	Tech Stack Summary	8
10.	Model Performance Comparison	8
11.	Working Application Link	9

1. PROJECT OVERVIEW

ExoHabitAI is an end-to-end machine learning web application that predicts the habitability potential of exoplanets using data from the NASA Exoplanet Archive. The system accepts planetary and stellar parameters as input and returns an AI-powered habitability score along with a classification label. Built using a Flask REST API backend and a space-themed HTML/CSS/JS frontend, the application is fully deployed on Render's cloud platform.

Dataset	NASA Exoplanet Archive — 6,100 confirmed exoplanets
Target	habitable_binary (0 = Non-Habitable, 1 = Potentially Habitable)
Model	XGBoost Classifier within a scikit-learn Pipeline
F1 Score	1.0 (Perfect) ROC-AUC: 1.0
Class Balance	27 habitable out of 6,100 planets (0.44%)
Live URL	https://b13-exohabitai-1b2y.onrender.com

2. MODULE 7 — VISUALIZATION & DASHBOARD

Module 7 delivers visual insights into the exoplanet dataset and model predictions. All visualizations are rendered dynamically in the browser using SVG, CSS animations, and the Fetch API — no external charting libraries required.

2.1 Dashboard Components

- **Habitability Gauge** — Animated SVG arc showing predicted score (0–100%) with dynamic color: green (>80%), gold (50–80%), orange (20–50%), red (<20%).
- **Probability Bars** — Animated bars showing model confidence for Habitable vs Non-Habitable with percentage labels.
- **Feature Importance Chart** — Bar chart showing the top 7 predictive features with contribution percentages, rendered on each prediction.
- **Rankings Table** — Full catalogue of 6,100 exoplanets ranked by ML-predicted habitability probability with configurable filters.
- **Top 3 Podium Cards** — Visual highlight of the three most habitable candidate planets with medal indicators.
- **Habitable Zone Badge** — Indicator showing whether the planet's equilibrium temperature is within the 180–310 K habitable zone.
- **Model Comparison Table** — Static table in the Model tab comparing all trained models across 5 evaluation metrics.

2.2 Feature Importance (from Random Forest Analysis)

Rank	Feature	Importance	Description
1	Habitability Score	49.3%	Custom engineered composite planetary-stellar index
2	Equilibrium Temperature	21.5%	Surface temperature (K) — raw value
3	Eq. Temp (Scaled)	19.7%	StandardScaler-normalized equilibrium temperature
4	Planet Radius (Scaled)	2.7%	Normalized radius in Earth radii
5	Jupiter Radius	2.5%	Radius expressed in Jupiter radii
6	Star Temperature	1.7%	Host star effective temperature (Kelvin)
7	Planet Radius	1.6%	Raw radius in Earth radii

■ *Habitability Score dominates predictions at 49.3%, confirming that domain-specific composite metrics outperform individual raw measurements in exoplanet habitability classification.*

3. MODULE 8 — DEPLOYMENT & DOCUMENTATION

Module 8 covers the full deployment pipeline from a local development environment to a live cloud-hosted application. Deployment uses Render's free tier with automatic redeloys triggered on every push to the ShishirTS branch of the GitHub repository.

3.1 Deployment Architecture

- **Host Platform:** Render Web Service (Free Tier) — Singapore region.
- **Runtime:** Python 3.10 — pinned via PYTHON_VERSION environment variable in Render settings.
- **Build Command:** `pip install --only-binary=:all: -r requirements.txt` — forces prebuilt wheels, avoids GCC compilation errors.
- **Start Command:** `gunicorn backend.app:app` — production WSGI server.
- **Auto-Deploy:** Every push to the ShishirTS branch triggers an automatic redeploy on Render.
- **Static Files:** Flask serves the frontend (index.html, style.css, script.js) directly from the `frontend/` directory.

4. FLASK BACKEND ARCHITECTURE

The Flask backend serves a dual purpose: it acts as a REST API for ML predictions and simultaneously serves the static frontend files. This single-service architecture eliminates CORS complexity in production and simplifies deployment to a single Render web service.

4.1 File Structure

- **backend/app.py** — Main Flask application: all API routes, static file serving, model loading.
- **backend/utlis.py** — Helper utilities: input validation, HZ temperature check, feature metadata.
- **models/final_model.pkl** — Serialized XGBoost Pipeline loaded at startup via joblib.
- **data/processed/preprocessed.csv** — Source dataset used by `/rank` and `/stats` endpoints.
- **data/processed/habitability_ranked.csv** — Pre-computed habitability rankings.
- **frontend/** — `index.html`, `style.css`, `script.js` served statically by Flask.

4.2 Model Loading and Error Handling

The model is loaded once at application startup using joblib. If the model file is missing or has a version mismatch, the application starts in degraded mode — the health endpoint returns `model_loaded: false` and all prediction endpoints return HTTP 503. This prevents silent failures in production and gives clear diagnostic information.

4.3 Input Processing Pipeline

User inputs are validated and converted into a pandas DataFrame matching the exact column structure the model was trained on. Missing optional fields (scaled columns) are filled with NaN —

the pipeline's SimpleImputer handles these automatically using median values from training data.

- **Required fields:** 13 parameters including radius, mass, orbital period, temperatures, and habitability indices.
- **Optional fields:** 9 pre-scaled columns — auto-imputed by the pipeline if not provided.
- **Star type flags:** 8 boolean columns auto-derived from the star_spectype spectral classification.
- **Validation:** Type checking, missing field detection, and descriptive error messages as JSON.

5. API ENDPOINTS REFERENCE

All API endpoints return structured JSON with a status field (success/error), relevant data, and descriptive error messages. CORS is enabled globally. In production, all routes are prefixed with /api to avoid conflicts with the frontend's static file routes.

5.1 Endpoint Summary

Method	Endpoint	Description	Status Codes
GET	/api/health	Model load status + pipeline steps	200
POST	/api/predict	Single planet habitability prediction	200 / 422 / 503
POST	/api/predict/batch	Batch prediction with automatic ranking	200 / 503
GET	/api/rank	Ranked planet catalogue (limit, filter)	200 / 404
GET	/api/features	Required fields + feature importance data	200
GET	/api/stats	Dataset overview statistics	200
GET	/	Serves frontend index.html	200

5.2 POST /api/predict — Request Fields

Field	Type	Example	Description
planet_name	string	Kepler-452b	Optional label for the planet
radius_earth	float	1.63	Radius in Earth radii
mass_earth	float	5.0	Mass in Earth masses
orbital_period	float	384.84	Orbital period in days
semimajor_axis	float	1.046	Semi-major axis in AU
eq_temp_k	float	265	Equilibrium temperature in Kelvin
density	float	5.5	Planet density in g/cm3
star_temp_k	float	5757	Host star temperature in Kelvin
star_luminosity	float	0.079	Star luminosity (log solar)
star_metallicity	float	0.21	Star metallicity [Fe/H]
habitability_score	float	0.61	Composite habitability index (0-1)

Field	Type	Example	Description
stellar_compatibility	float	0.9	Stellar influence index (0-2)
orbital_stability	float	5.2	Orbital stability metric
star_spectype	string	G2 V	Spectral type (e.g. G2 V, M3 V, K5)

5.3 POST /api/predict — Response Fields

Field	Type	Example Value
status	string	success
planet_name	string	Kepler-452b
prediction	int	1
label	string	Potentially Habitable
habitability_score	float	87.42
habitability_class	string	High
probabilities.habitable	float	87.42
probabilities.non_habitable	float	12.58

6. FRONTEND ARCHITECTURE

The frontend is a single-page application (SPA) with four navigation tabs: Predict, Rankings, Model, and About. It uses a deep-space aesthetic with an animated HTML5 Canvas starfield, CSS-animated orbiting planet system, and glassmorphism panel design. All styling is written in vanilla CSS with Bootstrap 5 for responsive layout.

6.1 Key UI Components

Component	Technology	Description
Starfield Canvas	HTML5 Canvas + JS	300 twinkling stars with sine-wave opacity animation
Planet System	CSS Keyframe Anim.	3 planets orbiting an animated gold star core
Habitability Gauge	SVG + JavaScript	Arc gauge with animated score counter, color-coded output
Prediction Form	HTML5 + Fetch API	14-field form with full client-side validation
Quick Fill	JavaScript	4 planet presets + 1 random fill with realistic ranges
Scan Overlay	CSS + JavaScript	Full-screen scanning animation during API processing
Rankings Table	Fetch API + HTML	Dynamic table from /api/rank with top-3 podium cards
Feature Bars	CSS Transitions	Animated importance bars on each prediction result

6.2 API Integration

- **Auto-detect environment:** Uses localhost:5000 in development, relative /api path in production.
- **Health polling:** Checks /api/health every 30 seconds and updates the status indicator in nav.
- **Error handling:** All fetch calls wrapped in try/catch with user-friendly messages.
- **Random fill:** Generates randomised planetary parameters within realistic astrophysical bounds.

7. ML MODEL INTEGRATION

The machine learning model is a scikit-learn Pipeline consisting of a ColumnTransformer preprocessor followed by an XGBoost classifier. The entire pipeline is serialized as a single .pkl file using joblib, ensuring preprocessing and prediction are atomic operations with no risk of data leakage.

7.1 Pipeline Architecture

Step	Component	Configuration
1. Imputation	SimpleImputer (median)	Handles 4,559 missing values across 23 numeric columns
2. Scaling	StandardScaler	Z-score normalization applied to all numeric features
3. Encoding	OneHotEncoder	handle_unknown='ignore' — encodes star_spectype categorical
4. Prediction	XGBClassifier	n_estimators=400, max_depth=10, scale_pos_weight=220

7.2 Class Imbalance Handling

The dataset contains only 27 habitable planets out of 6,100 (0.44%), a severe class imbalance. The scale_pos_weight parameter is set to 220 (= 4,858 / 22 training habitable planets), instructing XGBoost to penalise misclassification of habitable planets 220x more heavily. This maximises recall for the minority class — critical for scientific screening where missing a habitable candidate is far costlier than a false positive.

8. DEPLOYMENT ON RENDER

8.1 Configuration Summary

Service Type	Web Service — Python runtime
Region	Singapore (Southeast Asia) — lowest latency from India
Branch	ShishirTS — auto-deploys on every push
Build Command	pip install --only-binary=:all: -r requirements.txt
Start Command	gunicorn backend.app:app
Python Version	3.10.0 (set via PYTHON_VERSION environment variable)
Instance Type	Free — 512 MB RAM, shared CPU
Auto-Deploy	Enabled — triggered on every GitHub push

Spin Down

After 15 minutes of inactivity (free tier — first request may take ~50s)

8.2 Production requirements.txt

Package	Version	Purpose
flask	3.0.3	Web framework and static file serving
flask-cors	4.0.1	Cross-Origin Resource Sharing headers
joblib	1.4.2	Model serialization and loading
numpy	2.2.6	Numerical arrays — latest wheel for Python 3.10
pandas	2.2.3	DataFrame operations for model input building
scikit-learn	1.5.1	Pipeline, ColumnTransformer, preprocessors
xgboost	2.0.3	XGBoost gradient boosted classifier
gunicorn	22.0.0	Production WSGI HTTP server
scipy	1.13.1	Scientific computing (xgboost dependency)

■ *scikit-learn must be exactly 1.5.1. The model was serialized with this version — newer versions cause a `_RemainderColsList AttributeError` on deserialization.*

9. TECH STACK SUMMARY

Layer	Technology	Version	Role
ML Model	XGBoost	2.0.3	Gradient boosted binary classification
ML Pipeline	scikit-learn	1.5.1	End-to-end preprocessing + model pipeline
Data	pandas	2.2.3	DataFrame manipulation and model input
Numerics	numpy	2.2.6	Array operations
Serialization	joblib	1.4.2	Model save and load (.pkl)
Backend	Flask	3.0.3	REST API and static frontend serving
CORS	flask-cors	4.0.1	Cross-origin request headers
WSGI Server	gunicorn	22.0.0	Production HTTP server
Frontend	HTML5 / CSS3 / JS	Vanilla	SPA with 4 tabs and animated components
UI Framework	Bootstrap	5.3	Responsive grid and layout utilities
Fonts	Orbitron / Inter	Google Fonts	Display and body typography
Version Ctrl	Git / GitHub	—	Source control and CI/CD trigger
Deployment	Render	Free Tier	Cloud web service hosting
Language	Python	3.10	Backend and ML runtime

10. MODEL PERFORMANCE COMPARISON

All models were evaluated on a stratified 80/20 train-test split with 5-fold cross-validation. RandomizedSearchCV was used for hyperparameter tuning (40 iterations per model). Selection criteria prioritised F1-Score and Recall due to the 0.44% class imbalance.

10.1 Evaluation Results

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9689	0.1163	1.0000	0.2083	0.9865
Decision Tree (depth=5)	0.9994	1.0000	0.8636	0.9262	0.7992
Random Forest (tuned)	0.9992	0.8333	1.0000	0.9091	0.9998
XGBoost (final) ★	1.0000	1.0000	1.0000	1.0000	1.0000

■ XGBoost selected as the final model. No overfitting detected — training F1 (0.978) and test F1 (1.0) are consistent. Model saved as `models/final_model.pkl` using `joblib`.

10.2 Best Hyperparameters — RandomizedSearchCV

Hyperparameter	XGBoost (Best)	Random Forest (Best)
n_estimators	400	200
max_depth	10	None (unlimited)
learning_rate	0.1	N/A
subsample	0.8	N/A
colsample_bytree	0.7	N/A
scale_pos_weight	220.82	N/A (class_weight='balanced')
max_features	N/A	0.5 (sqrt)
CV Strategy	5-fold Stratified KFold	5-fold Stratified KFold
Best CV F1 Score	0.931	0.914

11. WORKING APPLICATION LINK

The ExoHabitAI application is fully deployed and accessible via the URL below. All four milestone modules are integrated: ML prediction engine, Flask REST API, space-themed frontend, and the rankings dashboard.

LIVE APPLICATION URL
https://b13-exohabitai-1b2y.onrender.com

11.1 Features Available at Live URL

- **Predict Tab** — Enter planetary parameters or use quick-fill presets to get instant habitability predictions with animated gauge and probability bars.
- **Rankings Tab** — Browse top habitable candidate planets from the NASA catalogue with configurable limit and filters.
- **Model Tab** — Why XGBoost was selected, 8-step training walkthrough, feature importance chart, and full model comparison table.
- **About Tab** — Project overview, complete API reference documentation, and tech stack.
- **Random Fill Button** — Generates randomised planetary parameters within realistic astrophysical bounds for quick testing.

11.2 Live API Endpoints

Endpoint	Description
/api/health	API health check — confirms model is loaded
/api/predict	POST — single planet habitability prediction
/api/rank?limit=10	Top 10 most habitable candidate planets
/api/stats	Dataset overview statistics
/api/features	Feature list and importance data

■ The free Render tier spins down after 15 minutes of inactivity. The first request after a period of inactivity may take up to 50 seconds. This is a platform limitation and does not affect application functionality.

ExoHabitAI — Predicting Life Beyond Earth

NASA Exoplanet Archive · XGBoost ML Pipeline · Flask REST API · Render Deployment